Exercícios — Aula 07

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

Continue na `api-filmes` refatorada em camadas na aula 06. Se a sua não ficou pronta, use a solução de `aula-06/exercicios/solucoes/api-filmes` como ponto de partida.

Antes de começar, ajuste os scripts do `package.json` (a flag é necessária no Node 22 e inofensiva no 24+):

```
"scripts": {
  "dev": "node --watch --experimental-sqlite --env-file=.env src/server.js",
  "popular": "node --experimental-sqlite --env-file=.env src/banco/popular.js"
}
```

Nível 1 — obrigatório

Exercício 1 — SQL na unha (sem Node)

Antes de programar, escreva os comandos SQL em um arquivo `consultas.sql`. Só o texto do SQL, para treinar a sintaxe:

1. Criar a tabela `filmes` com: `id` (PK autoincrement), `titulo` (texto obrigatório), `ano` (inteiro obrigatório), `genero` (texto obrigatório), `nota` (real, padrão 0), `assistido` (inteiro, padrão 0), `criado_em` (texto obrigatório).
2. Inserir três filmes.
3. Buscar todos os filmes de drama.
4. Buscar os filmes com nota maior ou igual a 8, do mais bem avaliado para o menos.
5. Buscar filmes cujo título contenha "de" (dica: `LIKE '%de%'`).
6. Marcar o filme de id 2 como assistido.
7. Apagar o filme de id 3.
8. Contar quantos filmes existem por gênero (dica: `GROUP BY`).

Exercício 2 — Estrutura do banco

Crie:

- `src/banco/schema.sql` — o `CREATE TABLE` do exercício 1, com `IF NOT EXISTS`.
- `src/banco/conexao.js` — abre o banco (caminho vindo do `.env`), liga `PRAGMA foreign_keys = ON` e roda o schema.

- `src/banco/popular.js` — insere 5 filmes de exemplo. Deve poder ser rodado várias vezes sem duplicar dados.

Adicione `*.db` ao `.gitignore`.

Exercício 3 — Repository com SQLite

Reescreva **só** `src/repositories/filme.repository.js` usando o banco. Mantenha exatamente os mesmos nomes de método e o mesmo contrato de retorno da versão em memória.

Cuidados obrigatórios:

- Toda consulta com valor de fora usa `?`. **Nenhuma exceção.**
- Uma função `paraObjeto(linha)` traduz `snake_case` → `camelCase` e `0/1` → `false/true`.
- `atualizar` devolve `null` quando `changes === 0`.
- `remover` devolve `true` / `false` conforme `changes > 0`.

A prova de que deu certo: rode todo o seu `api.http`. O resultado deve ser idêntico ao da aula 06. Nenhum outro arquivo pode ter sido modificado — confira com `git status`.

Exercício 4 — Persistência de verdade

1. Suba a API e crie um filme novo.
2. Pare o servidor (`Ctrl+C`).
3. Suba de novo e liste.

O filme continua lá? Então o objetivo do módulo de back-end foi cumprido. 🎉

Nível 2 — esperado

Exercício 5 — Filtros no SQL, não em JavaScript

Hoje o service traz **todos** os filmes e filtra com `.filter()`. Com 10 registros, tudo bem. Com 100 mil, é inaceitável — você carrega o banco inteiro na memória para jogar quase tudo fora.

Mova a filtragem para o banco. No repository, crie:

```
buscarComFiltros({ genero, assistido, notaMinima, busca }) {
  const condicoes = [];
  const valores = [];

  if (genero) {
    condicoes.push("genero = ?");
    valores.push(genero);
  }
  if (busca) {
    condicoes.push("LOWER(titulo) LIKE LOWER(?)");
    valores.push(`%${busca}%`);
  }
  // ... os outros filtros

  const onde = condicoes.length > 0 ? `WHERE ${condicoes.join(" AND ")}` : "";
  return db.prepare(`SELECT * FROM filmes ${onde} ORDER BY
        id`).all(...valores).map(paraObjeto);
}
```

⚠️ Repare: os **nomes de coluna** são fixos, escritos por você. Só os **valores** entram por `?`. Essa distinção é o que mantém a consulta segura.

Exercício 6 — Tabela de avaliações com chave estrangeira

```
CREATE TABLE IF NOT EXISTS avaliacoes (
  id         INTEGER PRIMARY KEY AUTOINCREMENT,
  filme_id   INTEGER NOT NULL,
  autor      TEXT    NOT NULL,
  nota       INTEGER NOT NULL,
  comentario TEXT,
  criada_em  TEXT    NOT NULL,
  FOREIGN KEY (filme_id) REFERENCES filmes(id) ON DELETE CASCADE
);
```

Migre o repository de avaliações para o banco e **teste a integridade**:

1. Tente criar uma avaliação com `filme_id` que não existe → o banco deve recusar. (Se não recusar, você esqueceu o `PRAGMA foreign_keys = ON`.)
2. Apague um filme que tem avaliações → elas devem sumir junto, por causa do `ON DELETE CASCADE`.

Depois, escreva a média das notas **em SQL**, com `AVG()`, em vez de calcular em JavaScript.

Desafio — opcional

Exercício 7 — Consultas com JOIN

Crie `GET /filmes/:id/completo`, que devolve o filme com suas avaliações embutidas:

```
{
  "id": 1,
  "titulo": "Cidade de Deus",
  "notaMedia": 9.3,
  "totalAvaliacoes": 2,
  "avaliacoes": [ { "autor": "Ana", "nota": 5 }, ... ]
}
```

Faça de duas formas e **compare**:

1. Duas consultas separadas (uma no filme, outra nas avaliações).
2. Uma consulta só com `JOIN`.

Escreva num comentário qual você acha mais legível e por quê. Não existe resposta única — a discussão é o exercício.

Exercício 8 — Transações

Uma **transação** garante que um conjunto de operações aconteça por inteiro ou não aconteça:

```
db.exec("BEGIN TRANSACTION");
try {
  // várias operações...
  db.exec("COMMIT");
} catch (erro) {
  db.exec("ROLLBACK");   // desfaz tudo
  throw erro;
}
```

Crie `POST /filmes/importar`, que recebe um array de filmes e insere todos. Se **qualquer um** falhar na validação, nenhum deve ser inserido.

Teste enviando 3 filmes, sendo o segundo inválido, e confirme que nenhum entrou no banco. Essa propriedade — tudo ou nada — é o "A" de ACID, e é uma das razões de existirem bancos de dados.

Checklist de entrega

- [ ] Todos os valores externos entram por `?` — **procurei por template literals no SQL e não achei nenhum**
- [ ] Nenhum `UPDATE` ou `DELETE` meu está sem `WHERE`
- [ ] `*.db` está no `.gitignore`; `schema.sql` e `popular.js` estão versionados
- [ ] Só o `repository` mudou em relação à aula 06 (confirmei com `git status`)
- [ ] Reiniciei o servidor e os dados continuaram lá
- [ ] `npm run popular` pode ser rodado duas vezes sem duplicar dados
